Machine Learning for Civil Engineering

Semester Project Instructions

Classification of number of lanes, transition areas and crossing from point clouds

Classes:

- 2 lanes
- 3 lanes
- 4 lanes
- Crossing
- Lanes transition
- Median >2m 4 lanes
- Median >2m 6 lanes

General Information:

The data used for this project topic is point cloud data captured using airborne laser scanning. This single point clouds represent blocks, that were cut out of a complete point cloud of the earth surface, by applying a filtering scheme using geospatial data of the road network. This procedure allowed to orient the road blocks uniformly with the driving direction oriented along the y axis. Further preprocessing steps involved a ground separation filter, that was used to remove above ground point of vegetation, vehicles and road furniture along the road. The RGB color values on the positions 3 to 6 (indexing from 0) were interpolated from a different point cloud generated using areal photogrammetry. Hence, the color values on the road surface may be shaded in areas with high vegetation around it. The most important factor apart from the geometry is most likely the intensity, which is the strength of the returned laser beam from the scanner.

Task:

1. **Conceptualization**
    a. The first thing you want to think about is the way you want to encode your data for inputting it into a model. There are several possibilities for this, as you may have already seen in exercise 4 on the topic of tree-based methods, one could encode a point cloud into a 1-dimensional feature vector by simply taking the feature means off all points for each point cloud. While this can be done in theory, it is far from a sophisticated approach, since many feature´s means won´t be very characteristic and vary much over different point clouds. However, if you find characteristic features, that are able to describe the whole point cloud regarding the target variable, the results for this approach could be significant.
    b. A second possibility for encoding the point cloud would to convert the respective point clouds into an image representation, this can be done, since we are looking at only the road surface, which is very close to a plane. You could construct an

occupancy grid in the xy-plane and thereby generate a 2-D representation of the 3-dimensional point cloud. These images can then be the input for standard convolutional neural networks, you have learned about in the lecture and exercise. This approach loosely follows early approaches from point cloud segmentation, which can be found e.g. on google scholar with the keywords: “birds eye view” + “point cloud segmentation” if you are interested to go further into detail.

c. The third approach would be try and directly input the whole point cloud into a model, in this case however you will need to write a custom data loader, which first of all handles the different sizes & and numbers of point in your input point clouds. This approach is more complex than the other two.

There are more than those approaches that are possible to be implemented, so these three shall be first ideas of potential concepts. It would be very interesting, if you choose multiple approaches and compare their performance as well, since the first one is really doesn´t require much work, if you only choose this one, we would want to see that you put more effort into feature extraction, which in turn requires you getting into point clouds more deeply.

2. Model selection

a. For the model you use, we don´t want to make any restrictions, so maybe start out with the simple encoding approach and a simple sklearn model, and then go into more depth with a customized model. For using encoding approach 2 it would be interesting to employ standard CNN classification models, that you can either implement yourselves with pytorch or tensorflow or choose a classification model from the literature.

3. Data Pre-processing

a. The data might involve road blocks of different sizes, so keep that in mind when encoding features and potentially writing a data loader.

b. Also the number of samples for each class is not 100% balanced you could also evaluate the impact of different dataset compositions, by first choosing all samples and secondly choosing only a balanced subsample.

4. Feature extraction

a. Depending on the encoding approach you choose, your feature extraction varies, since you could try and engineer per point cloud features, per point features or per ‘pixel’ features for the 2D image approach. Think about this before starting with feature engineering

5. Evaluating model performance

a. Evaluate your final model with a threeway split of training-set, validation-set and test-set. Since the validation-set will be used multiple times, while you optimize your models hyperparameters, this data will not be independent from your model at the end, this is why you will need the third test-set to really have some data, that was completely out of the whole training & optimization process to evaluate your model properly. A typical split would be 80%-train, 15%-validation, 5%-test.

b. Common Metrics: The common metrics we would suggest you to use are **overall accuracy**, **intersection over union** (similar to jaccard score, but depending on configuration) and of course **precision** and **recall**.

6. **Optimize hyperparameters & optimize features iteratively**
    a. For hyperparameter tuning have a look at grid search, although you could also construct it on your own with some nested loops to get a better overview of the models you generate.
    b. Often times it can help to review the own feature extraction process to optimize features after the first results have been generated. In most case the engineer learns more about the topics while trying to create a model that fits his required task.

These are just suggestions and we don´t want to restrict you towards a single workflow, so feel free to test out different things and just ask us if you are having problems.